

Table of Contents

Executive Summary	3
Design Approach	4
Flow charts/UMLs	4
Object Interaction Diagram	4
Use Case Diagram	5
User Stories	6
User Story (Purchase Items)	7
User Story (Stock items)	8
User Story (Create Warehouse)	8
Class Diagram	9
Sequence Diagram	9
Tech Stack	10
Overall Design and Justifications	11
Appendix	12
Communication	12
Important Functions	12

Executive Summary

To automate the process of our warehouses, we have decided to go through with a centralized autonomous robot solution. The central system controls all our warehouses simultaneously, ensuring prompt and timely deliveries and constant stock of products.

Beginning with the layout of the warehouse, we have dedicated the borders purely for robot movement rather than for products. It allows the robots to avoid crashing against the shelves, which would cause more problems. The movement of the robots follows a rectangular method, always going East, North, West, South (E,N,W,S), which allows for synchronous and controlled movement. We stock through the stocking bays (North-East corner) of all warehouses and deliver through the delivery bays (North-West corners) of the warehouse. This allows for robots to move in a standard fashion.

For the robots moving, the warehouse computer holds the data of all the robots within the warehouse and their locations. The next robot in the stack checks if they have enough battery to reach their item. If so, the robot moves out of its docking position (South-West corner) to its item following the movement algorithm E,N,W,S. For robots accomplishing deliveries, they would grab the item from our shelf and go to the delivery truck and check and see if there is any space to fit the item. If not they would signal the delivery to submit its deliveries and stock it onto the next truck into the queue. Once placed into the truck, it would return to its docking station last in the stack to charge. For robots stocking items, it would again check its battery and grab an item from a stocking truck until the stocking truck is empty. It would return back to the docking location, top up its battery, and go to its item location to place it on its respective shelf. For both cases, the warehouse computer tells the robot whether it can move to its next step or wait for a robot that is in the way to move out of its desired location.

All the warehouses lead to one central computer that can be accessed by both a manager and our clients. After signing up, clients should be able to see the products in stock, add multiple items to a cart, and buy their products. Once they place an order of the product, robots will be sent immediately, one for each product they order until we run out of robots. Should a manager login, they can see the stock levels of all the products or individual ones, and restock the out-of-stock products. To ensure that all our products stay in stock, it sends stock on trucks of every product that is out-of-stock to their respective warehouses. Should there be no more space in the warehouse, the managers are also capable of creating new warehouses and stocking them with whatever items they want wherever they want (as long as it has space on the shelves). For deliveries, although trucks are sent automatically to complete deliveries when they are full, a manager can also send a truck out for delivery whenever they want to ensure prompt delivery of products.

This new system should allow for an efficiency increase in our delivery system, as we will be able to handle orders the instant they come in.

Design Approach

We separated the design into 2 major categories: server-side and client-side. The server side would contain all the backend code as well as database interfacing. The client-side would include the UI for all types of users and access the database for any required information that would be displayed.

The backend would handle all the different types of warehouses and how each would interact with each other. The hierarchy would be as follows: central computer, warehouses, and finally trucks/robots. The central computer would communicate with all the warehouses, querying the ones it needs and the warehouses would issue their respective commands. Any commands by the central computer would reference an overarching database. This would allow for correct commands to be sent to the correct warehouses.

The client would handle all user interface interactions and query calls. The front end would contain any required buttons or display useful information depending on the user. It would reference the database for its query calls, sending it to the backend to retrieve what it wants.

Flow charts/UMLs

Object Interaction Diagram

The figure below outlines the interactions between different objects in our system. It is intended to provide a broad overview of the overall system framework and illustrate the communication channels between objects.

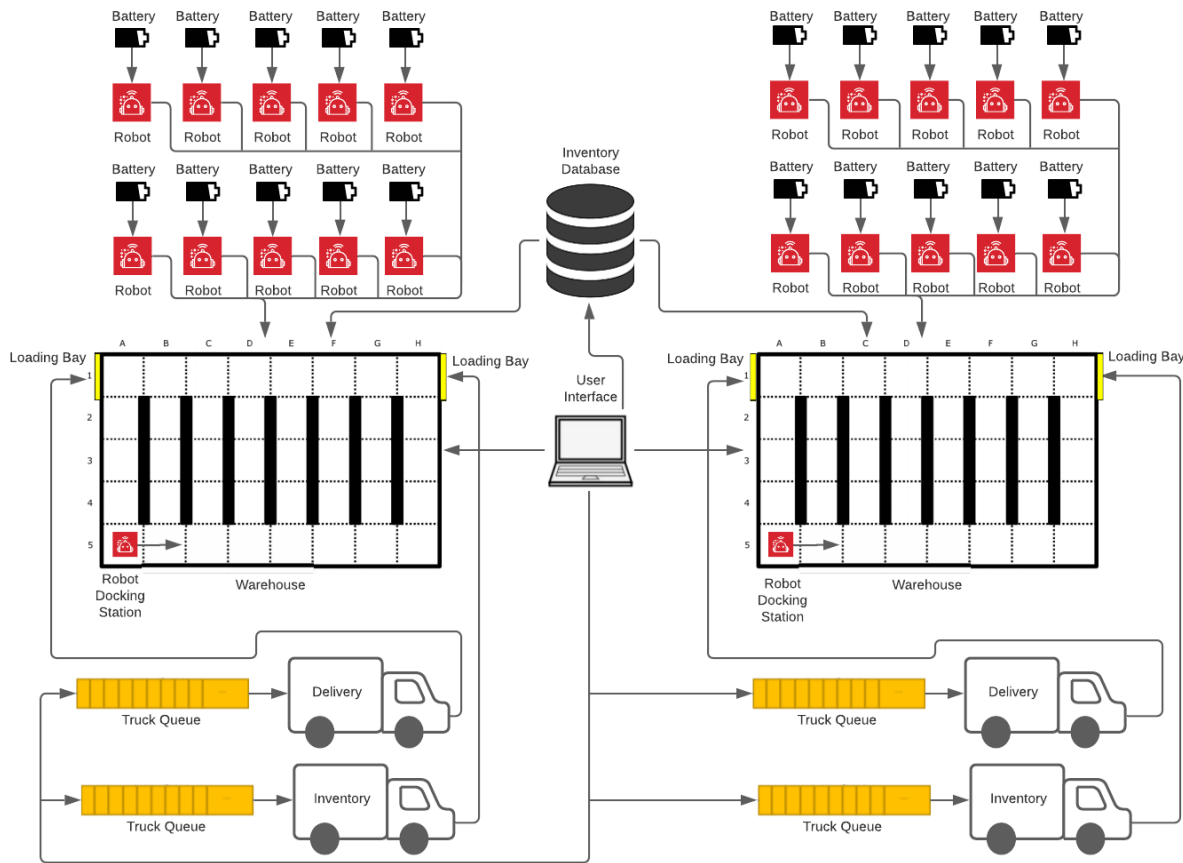


Figure 1. Object Interaction Diagram
[Full Diagram \(LucidChart\)](#)

Use Case Diagram

The following figure outlines how actors interact with our system to achieve specific goals. It details the requirements needed to achieve certain actions and aims to provide a conceptual framework of system operations from a user perspective.

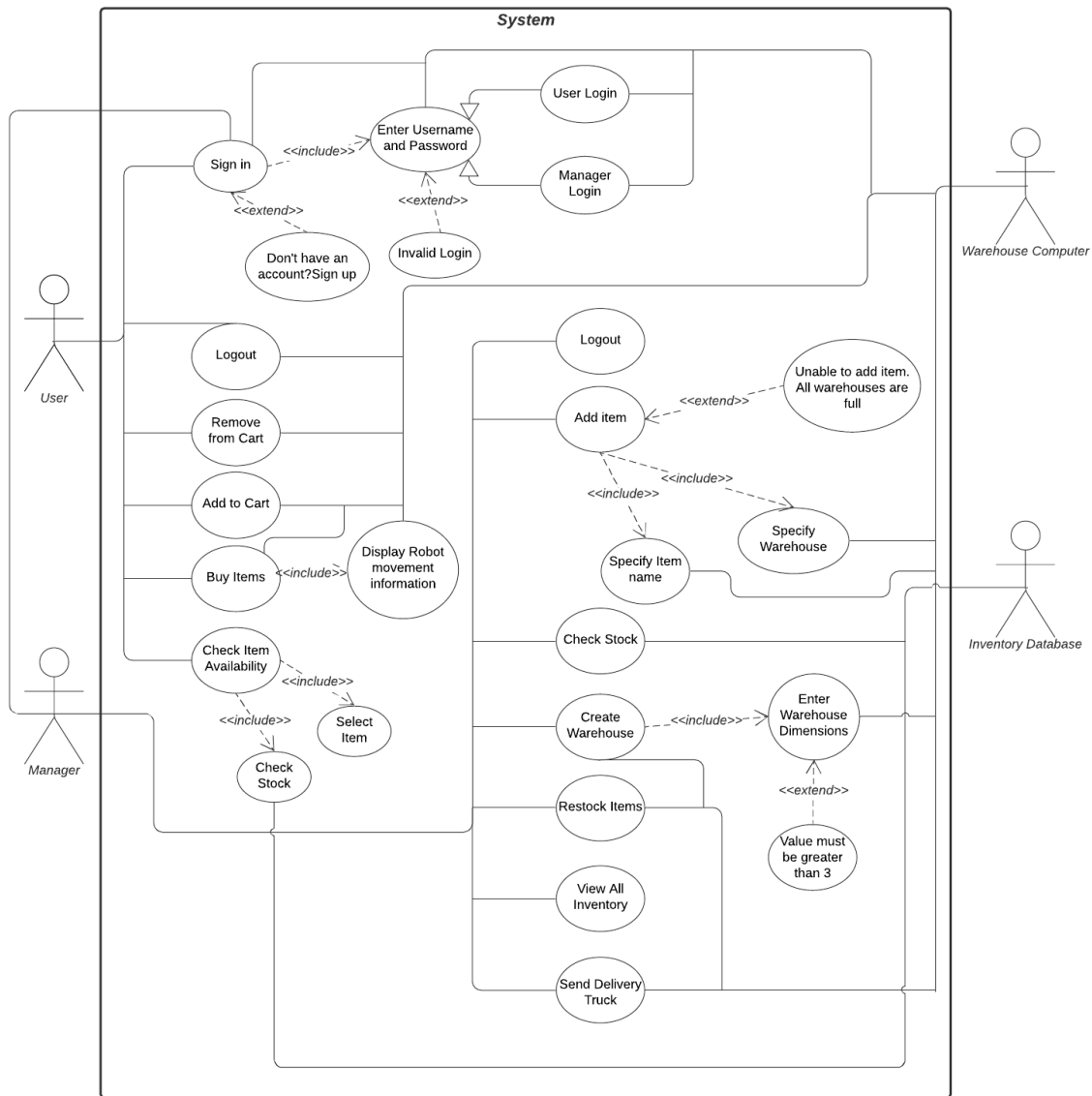


Figure 2. Use Case Diagram
[Full Diagram \(LucidChart\)](#)

User Stories

The User Stories below go into more detail about some of the possible interactions that can be seen in the Use Case Diagram above. They explain in a step-by-step manner how a user can navigate the interface and achieve various goals. User stories have been included for the following goals: Sign in, Purchase Items, Stock Items, and Create Warehouse.

Sign in

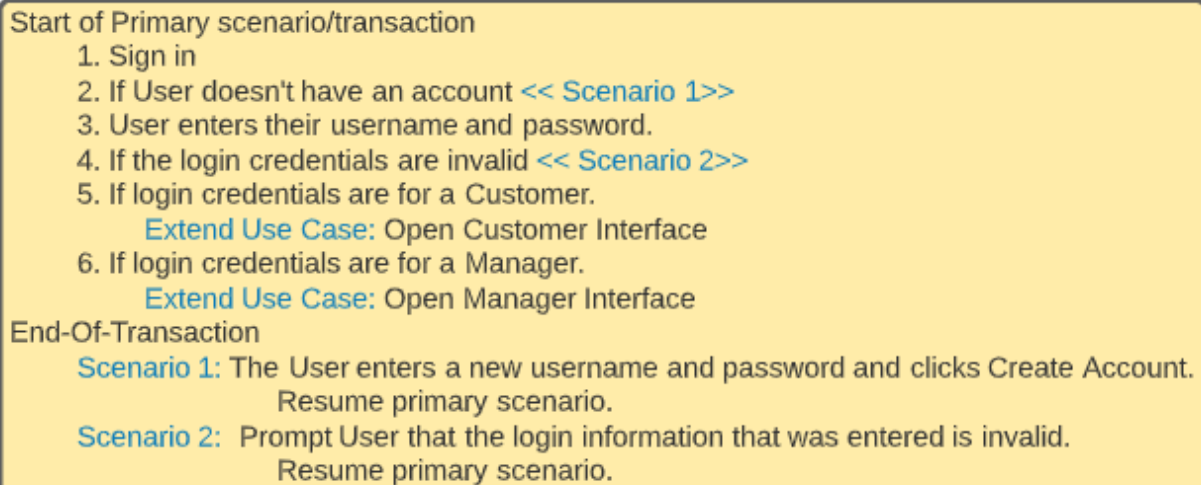


Figure 3. User Story (Sign in)
[Full Diagram \(LucidChart\)](#)

User Story (Purchase Items)

Purchase Items

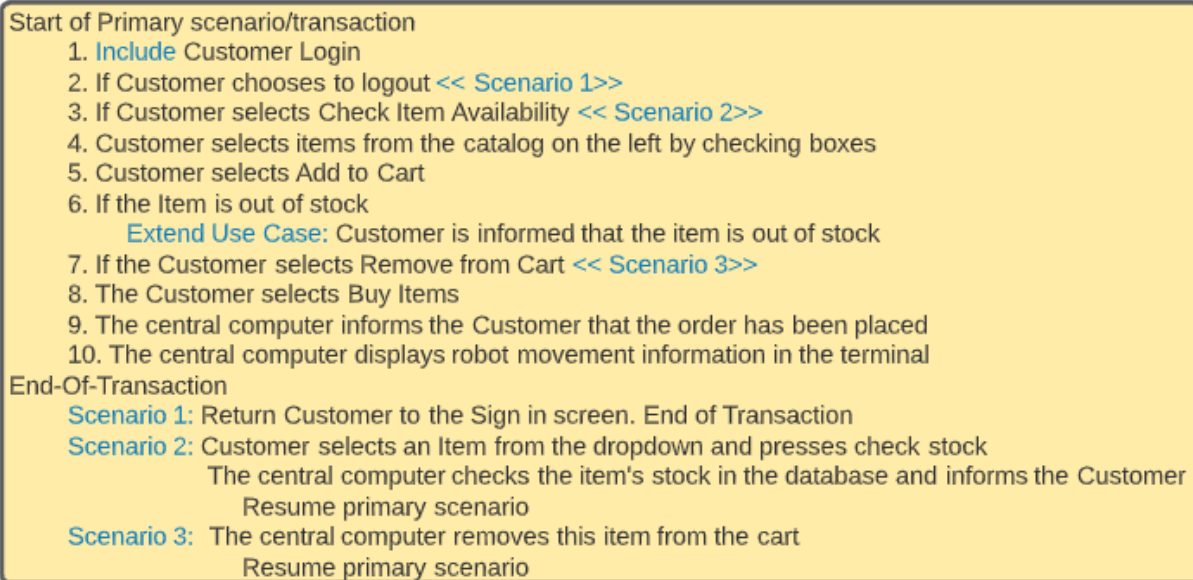


Figure 4. User Story (Purchase Items)
[Full Diagram \(LucidChart\)](#)

User Story (Stock items)

Stock Items

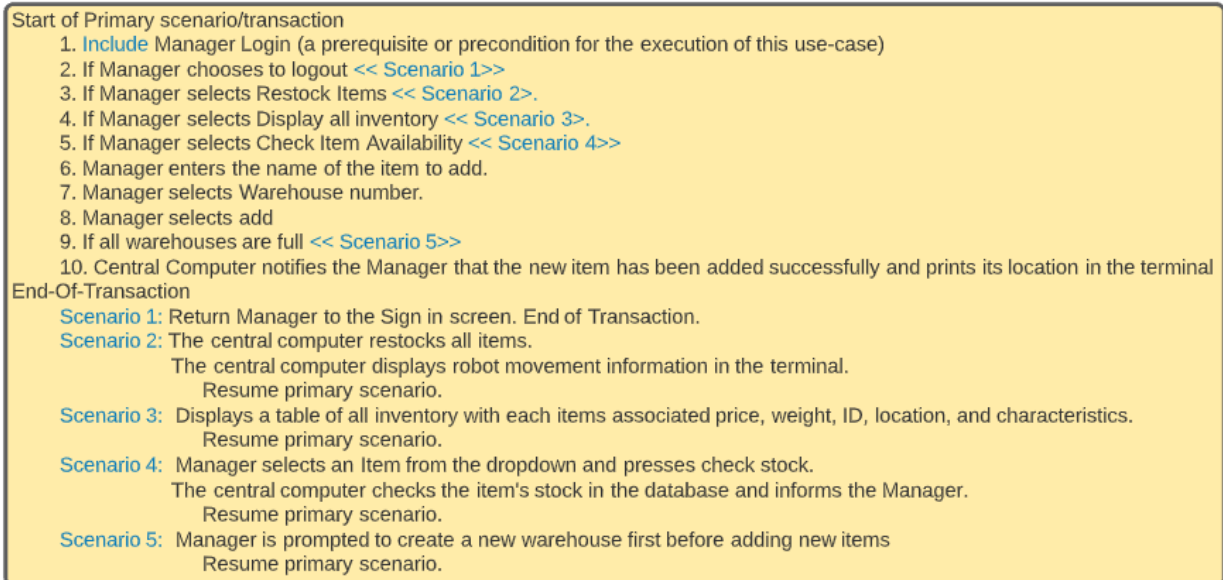


Figure 5. User Story (Stock items)

[Full Diagram \(LucidChart\)](#)

User Story (Create Warehouse)

Create Warehouse

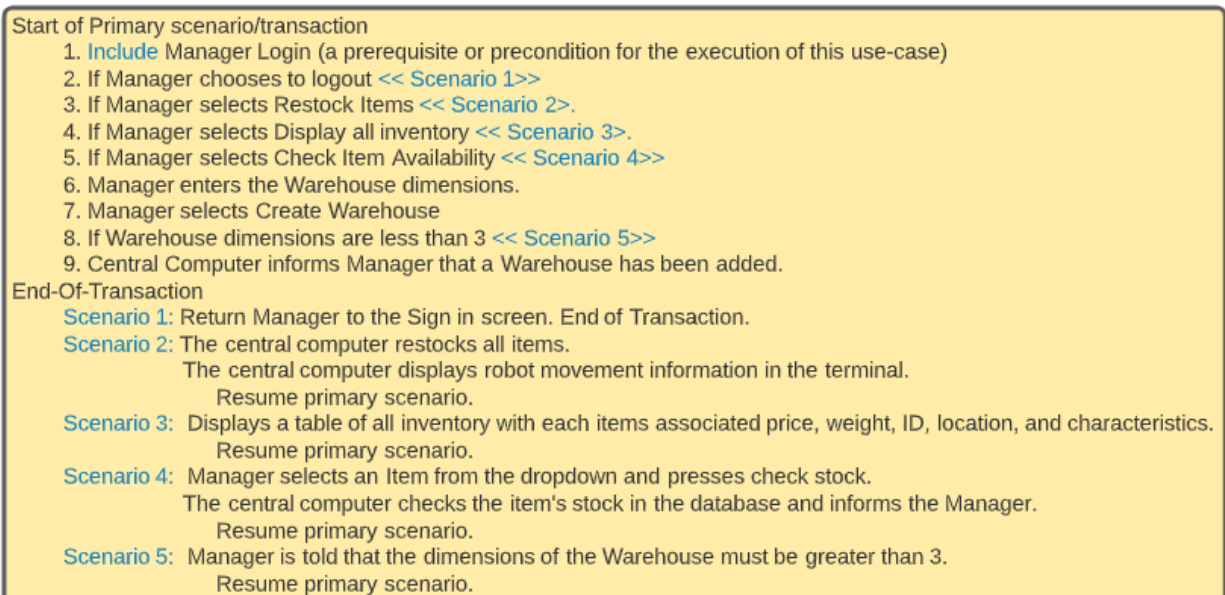


Figure 6. User Story (Create Warehouse)

[Full Diagram \(LucidChart\)](#)

Class Diagram

The figure below seeks to provide an overview of the classes within our system as well as the important member variables and functions. It also details the relationships between these classes and their associations and multiplicities.

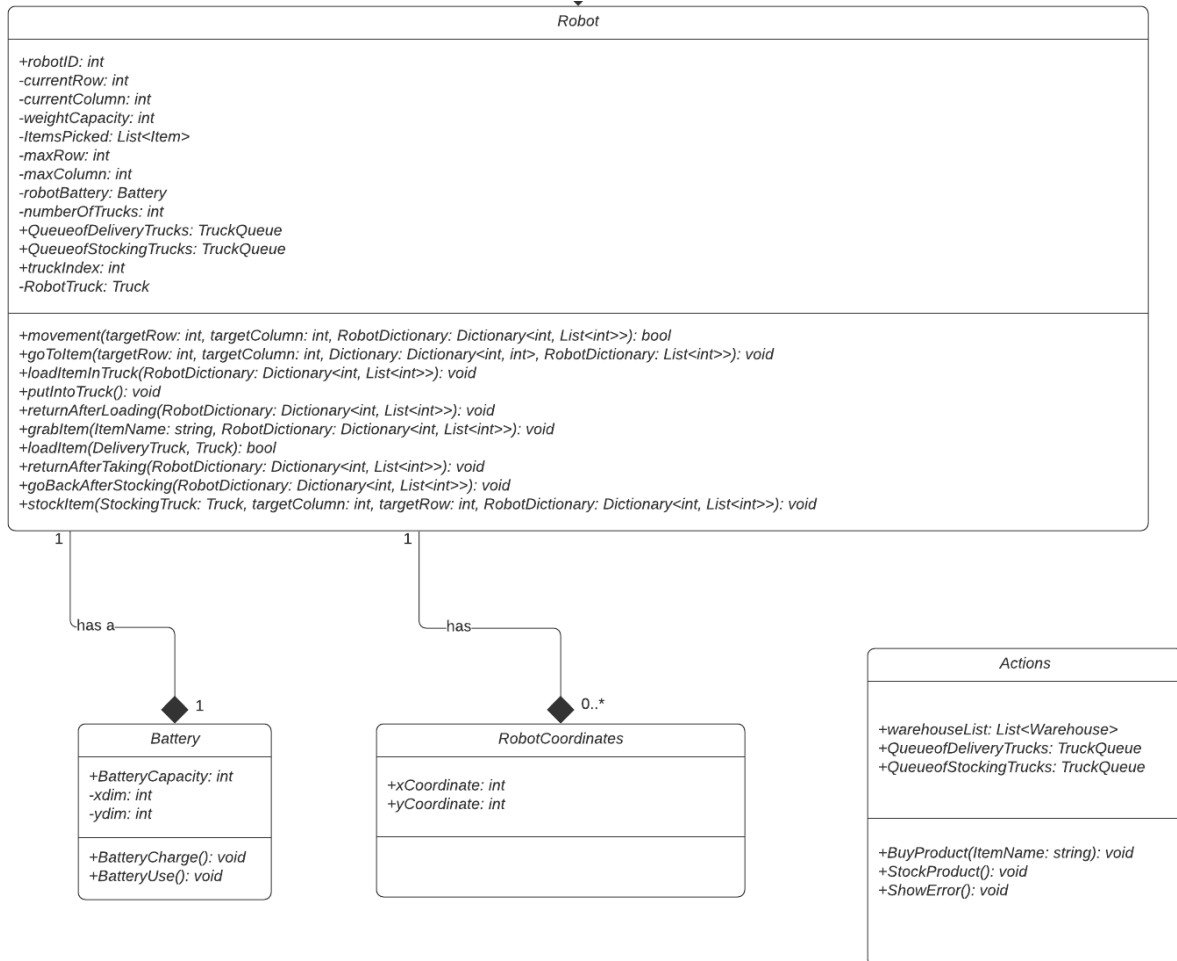


Figure 7. Class Diagram
[Full Diagram \(LucidChart\)](#)

Sequence Diagram

The following diagram is a step-by-step walkthrough of an example use case scenario initiated by the user. It outlines how objects pass messages to one another and the response time for the user. It aims to communicate the process of how typical user interactions might take place by looking into the internal signals being exchanged.

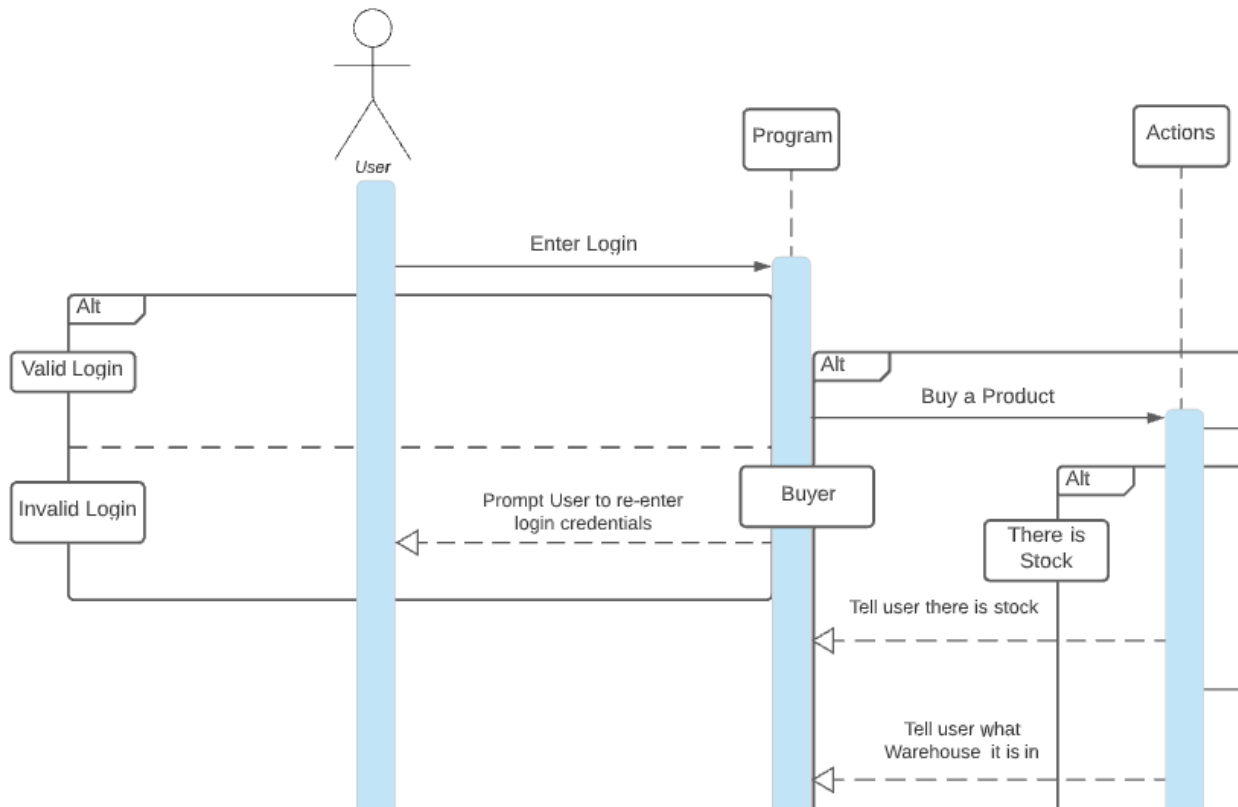


Figure 8. Sequence Diagram
[Full Diagram \(LucidChart\)](#)

Tech Stack

Application	Technology Used	Description
Front-End	WinForms	Event-based UI written in C#
Database	SQL	Local server to store warehouse and inventory data
Back-End	.NET	.Net framework to allow for multithreaded application

Table 1. Tech Stack

Overall Design and Justifications

Starting with the frontend we opted towards WinForms to have a .NET based UI that had event handlers. This choice was made as we could simulate multiple users and user types whilst keeping the interface of the backend and front end simple. It also allowed for direct calls to the backend as all of WinForms could be event-based. With the ability to have multiple users, we saw no need in having to make it a web interface as it would add an unnecessary layer of complexity to the project. WinForms also provided an easy to use interface as opposed to purely using a terminal. However, we did display certain information through the terminal rather than using WinForms as we did not want to clutter the interface.

Moving on to why we choose to use a database, having a local database allowed for object permanence for our data. That means that our items and warehouse would exist elsewhere and still be referenced rather than be created at start-up. This allows for easy storage of data and any changes made using the UI, to be reflected permanently until explicitly removed.

Addressing the backend, it can be separated into three major sections. The warehouse, the trucks and the robots. Starting with the warehouses, each warehouse would contain instances of trucks and robots respective to their own warehouse. To communicate between other warehouses the central computer (UI) would reference the database and see which warehouse has what and send respective commands to each warehouse for the warehouse computers to handle individually. The reason why we did this was that it allowed for concurrent orders to be placed and meant that the warehouses were independent of each other, capable of fulfilling orders as quickly as possible.

Moving onto the robots, we arbitrarily assigned each warehouse 10 robots and they are stored within a docking station where they are charging their battery. As soon as an order comes in the last robot in the stack checks if it has enough battery. It then checks if anything is occupying the next location if so it stays within the docking station. If all conditions pass, it gets popped out of the room and goes to the item location following E,N,W,S. The reason why we use a stack is because theoretically the robots are lined up and it allows the robot "at the door" to just leave the docking station to grab its respective item. With each movement, the warehouse computer checks the position of all the robots and tells the robot if there is something occupying its desired position. This is a critical part of the code as when we check to see it must be mutexed to make sure that it isn't being edited at the same time we are checking. If it is occupied, wait until the robot moves if not continue on with its action and then update the dictionary. Again this is a critical part of the code that must be mutexed again as it is editing a shared resource and no other robots should have access to it as it is editing. It then either puts the item on the shelf or puts it into a delivery truck depending on whether it is stocking or

fulfilling an order. After its action, it returns back to the stack and charges as it waits for its next instruction.

Finally, with the trucks, there are two types being stocking and delivery trucks, 10 of each for each warehouse. This allows for simpler movement of the robots as well as simplifying the queues as it will not need to distinguish what type of truck it is in within the queue. With each respective loading bay, the trucks check if they are full capacity in volume and or weight. If so they either send for restocking or delivery and enter back at the end of the queue. They can also be sent manually by the manager so the trucks are not waiting infinitely if no actions come in.

Appendix

Communication

In order to ensure that the warehouse computers, central computers, and robots communicate properly, it follows the following procedures.

Central Computer -> Warehouses

- The central computer connects to the warehouses and sends queries to a SQL database on the central computer.
- Central computer handles all customer and manager queries and sends warehouses respective commands

Warehouses -> Robots/Trucks

- The robots get commands from the warehouses to move and add to the trucks
- The trucks are sent to delivery automatically if they are full but the warehouse could also send trucks manually
- The warehouse computer keeps the location of all the robots and tells robots how to move

Robots -> Battery

- Robots keep information of battery and location and see if they can make it to the location

Important Functions

BuyProduct(string stringName)

- Starts a thread to send a robot to a stocking shelf and grabs the 'stringName' item from its location

StockItems()

- Starts threads to stock a truck and send robots to grab from truck and put it onto shelves

movement(int targetRow, int targetColumn, Dictionary<int, List<int>> RobotDictionary)

- Checks and sees if robot has enough battery to reach spot
- Charges robot and returns false if it cannot
- Sends robot to grab item and returns true if it can

createNewWarehouse(string Xvalue, string Yvalue)

- Adds warehouse to both database and warehouse list of size 'Xvalue' and 'Yvalue'
- `addProduct(int warehouseID, string itemName)`
- Adds product to the database and warehouse